Discovering and exploiting vulnerabilities in critical, real-world software with Claude

Published	Oct 27, 2026
Team	Frontier Red Team

Over the past few months, we've used an unreleased, internal research model to find critical vulnerabilities in some of the world's most widely used software, such as browsers and operating systems. In this post, we share details of five attack chains, which led to remote code execution and other critical impacts with little to no user interaction. These attack chains demonstrate the model's ability to create sophisticated, multi-step exploits in both open-source and proprietary software. Any vulnerabilities in these products could affect billions of users. They're also among the most security-critical software in the world, making them a bellwether for testing models' vulnerability research and cyber capabilities.

We reported each vulnerability to the affected vendor under our coordinated vulnerability disclosure, which gives developers 90 days to patch and a further 45 days before we discuss details beyond what is already included in a public advisory.

1. One-click remote code execution in Chrome

After initial prompting, Claude wrote a full exploit chain enabling an attacker to execute arbitrary code as the user, outside the browser's sandbox. Our proof of concept launches the Calculator app from a stock default-configuration Chrome browser on macOS within seconds of the page loading, without any further user interaction:

- Claude discovered a vulnerability in Chrome's JavaScript engine, V8, that allowed a malicious web page to corrupt V8's heap memory.

- Claude discovered a second vulnerability to break out of V8's memory region sandbox and defeat ASLR. This step located Chrome's code in memory and used a return-oriented programming (ROP) chain to take full control of the renderer process.

- The compromised renderer was confined by Chrome's process sandbox. Claude then discovered a race condition in CacheStorage. The compromised renderer used the race condition to read memory outside the sandbox and learn the memory layout of the privileged browser process.

- Finally, Claude discovered a use-after-free in FedCM, Chrome's federated sign-in feature, that allowed the exploit to execute arbitrary code outside the sandbox.

We have reported our findings to Google: 62 CVEs have been assigned and patched as of Chrome 156, with all of them rated critical or high severity. Among them are 13 sandbox escapes and 21 vulnerabilities that could have been used to compromise the renderer.

2. A one-click exploit chain in Safari

After initial prompting, Claude wrote an exploit chain that let any website's JavaScript completely bypass the same-origin policy (SOP). The exploit could access a user's Safari history, cookies, and other sensitive data, and could send authenticated HTTP requests to other domains and read the responses. For example, if the user is signed in to their email account, the attacker can now read their email; if they're logged into their bank account, the attacker can view account balances or attempt transfers.

To do this, Claude defeated several layers of protection built into Safari:

- Claude discovered a use-after-free (UAF) vulnerability in JavaScriptCore, Safari's JavaScript engine, which is written in C++. Certain TypedArray operations wrote through stale pointers to freed memory, overwriting TypedArray metadata.

- WebKit's Gigacage mitigates this type of attack by isolating TypedArray data in a separate region of memory. Claude discovered that certain variables in this region

were used to index into other regions, which gave it an arbitrary read/write primitive outside the Gigacage.

- All website JavaScript runs in the WebContent sandbox, which keeps bugs like the UAF from reaching sensitive browser data on disk. Claude discovered a vulnerability in the sandbox's IPC mechanism that let the exploit read Safari's stored history, cookies, and session databases from disk.

- Finally, Claude used the UAF to overwrite a variable in memory and disable the SOP, allowing the attacker's page to read data from any other origin.

The chain is interesting because it is a data-only attack that never relies on arbitrary code execution. Instead, it uses full read/write access to the browser process's memory to modify a security flag, an object size, and a URL. Several of the weaknesses are not vulnerabilities on their own but intended behaviors that become exploitable when combined.

Apple acknowledged our report the same day we submitted it and implemented a fix promptly. That fix is available to users.

3. Android lock screen bypass

After initial prompting, Claude wrote an exploit that let an attacker extract photos and other personal information from a stolen Android phone without unlocking it. The work combined three methods: source code analysis of the open-source parts of Android, reverse engineering and deobfuscation of the Google Play services binaries, and live testing on a physical Google Pixel 9. This turned up a wide range of vulnerabilities in both the open-source and proprietary components. Claude then chained three of them into the final exploit.

Android has several layers of protection against this attack. Claude found a way around each one:

- A locked Android phone is designed not to transfer data over USB. Claude found that a USB host with a specific setting bypassed this.

- The device-to-device transfer protocol refuses data commands from unauthorized peers. Claude found a way to skip the authorization check and start the pairing

process.

- Pairing is protected by an authenticated key agreement that asks the user to verify a six-digit code. Claude found a flaw that skipped this verification and behaved as if the user had tapped “Allow.”

- USB transfers are not supposed to carry file contents, as enforced by a hardcoded check. Claude found a fallback mechanism that sent raw bytes over USB and was not covered by the check. The exploit requested file contents through it.

This issue has been addressed by Google.

4. Windows remote denial of service

After initial prompting, Claude wrote an exploit that would have allowed an attacker to remotely crash any Windows machine that responds to HTTP/3 requests using the native Windows implementation. The difficulty in this case was in identifying the vulnerability in the first place. Following that initial prompting, we reported this vulnerability directly to Microsoft, which resolved the vulnerability and assigned it CVE-2026-33096.

The vulnerability that Claude identified was a memory safety out-of-bounds read in the Windows kernel’s implementation of QUIC. An attacker could reach the vulnerable codepath remotely over the network and send a few packets of data to a listening server to crash and restart the machine. By repeatedly sending these requests, it is possible to continuously crash the server, which causes loss of access to the server for all of the users. We estimate that there are likely several million such servers listening on the internet.

Claude discovered this vulnerability through binary analysis. We provided the model with a copy of the Windows executable and asked Claude to reverse-engineer how the software works using the existing reverse-engineering tools (such as Ghidra), write a fully annotated C program that reproduces the relevant features, compare the behavior reproduced program to binary to ensure correctness, and then study the code for vulnerabilities. When we asked Claude to identify potentially higher-risk issues, it surfaced this remotely exploitable vulnerability.

5. Zero-click remote code execution in Linux

While this post focuses on third-party products, our work on open source has continued. With Claude, we discovered local privilege escalation vulnerabilities on OpenBSD, FreeBSD, and Linux; a number of sandbox escapes in popular sandboxing infrastructure; and numerous vulnerabilities in important web applications. We have also found remotely triggerable (zero-click) arbitrary code execution vulnerabilities in the Linux kernel's networking stack, which have since been patched upstream and publicly disclosed. We detail one such example below.

After initial prompting, Claude wrote an exploit that allows a remote attacker to send packets to Linux servers with a multipath TCP listener to run arbitrary commands as root, without any credentials or user interaction. The underlying vulnerability is a race condition-induced UAF bug in the kernel networking stack. The full exploitation has two key stages:

- In the first stage, the attacker defeats KASLR remotely. It exploits the race to make the kernel read an attacker-controlled length field to copy extra bytes and send them back to the attacker, and as long as one such overread contains a kernel pointer in the reply, the attacker learns the kernel address space layout.

- In the second stage, the attacker hijacks the control flow. It exploits the same UAF bug to overflow a buffer and overwrite a callback pointer with the address of a payload the attacker has already staged in kernel memory via an ordinary protocol feature. When the kernel invokes the callback, the payload chains a series of calls to existing kernel functions. ending with an attacker-chosen shell command running as root.

An enabling property of this race condition is that a lost race only fails the handshake without crashing the whole system, so a determined attacker can retry indefinitely until they succeed. In our proof of concept, it takes 48 minutes to leak a kernel pointer and another five minutes to reach the shell command execution. This vulnerability (CVE-2026-98432) was patched upstream and backported to all maintained stable kernel branches.

What we learned

Throughout our process of using Claude for this security work, we identified a number of insights that shaped our approach. We think two are particularly important.

Test-time compute scaling and scanner diversity substantially increase how many vulnerabilities you find. We built a platform that runs large numbers of scanning agents in parallel, measures each scanner by how many additional vulnerabilities it contributes to the mix, and tracks coverage dynamically so the scanners process the entire codebase.

The strategies range from pure coverage (is there a vulnerability in this function?) to goal-based (does this app expose anything that could compromise customer data?) to threat model-based (does anything in this app bypass its documented threat model?). We also used Claude to run automated variant analysis on every confirmed finding. All findings pass through a standardized verification and risk rating pipeline, so we can experiment freely with new scanners without lowering the quality bar.

Fixing the existing backlog only solves half the problem; the other half is cutting the rate at which new vulnerabilities enter the codebase. For that, we recommend scanning at PR time. We built security guidance directly into our development process through a Claude Code plugin, which we've released publicly.

What this means for defenders

Frontier models have the ability to discover and combine multiple vulnerabilities into a working exploit. These skills have historically been the domain of few specialist teams with expertise in these targets. Now, it's something models can do with little human involvement, and then prove by producing a working exploit that anyone can run and verify.

Enabling defenders with powerful models is critical to giving them a headstart on patching systems before attackers do. Recently, we released an expanded version of our Cyber Verification Program, which makes advanced cyber capabilities and reduced blocking classifiers available to qualifying security professionals. It includes access to our most capable models, including Claude Opus 5.5, Claude Sonnet 5.5, Claude Mythos 5.1, and other new models moving forward. Our advice to defenders remains the same: scanning your own code is more important than ever.



Research

Policy

Commitments

Company

Products

Models

Solutions

Help and security

Terms and policies

Our mission

Anthropic is a Public Benefit Corporation. Our mission is to ensure the world safely makes the transition through transformative AI.

Log in

Claude.ai ↗

Claude console ↗

© 2026 Anthropic PBC